

# YOLOv8 vs YOLOv11 — Model Comparison

**Project:** AI-Based Intelligent Vehicle Monitoring and Traffic Violation Detection System  
**Module:** Vehicle Detection & Tracking (Member 1) **Test set:** 254 clips, UCSD/VISAL Traffic Database (day footage) **Models compared:** yolo8s (11.17M params) vs yolo11s (9.46M params) vs yolo11m (20.11M params) **Tracker:** ByteTrack (unchanged across all runs)

---

## Why We Tested This

Our mentor recommended switching from YOLOv8 to YOLOv11. Rather than switching based on YOLOv11 being newer, we ran a controlled comparison on our actual dataset to check whether it genuinely performed better for our specific problem before committing.

---

## General Architecture Comparison (Independent of Our Data)

| Aspect                     | YOLOv8                    | YOLOv11                                                                      |
|----------------------------|---------------------------|------------------------------------------------------------------------------|
| Release                    | Newer generation baseline | Ultralytics' latest iteration                                                |
| Design                     | Established backbone/neck | Refined backbone/neck, generally fewer parameters                            |
| Typical benchmark behavior | Strong, well-validated    | Often better efficiency (accuracy per parameter) on standard COCO benchmarks |
| General reputation         | Reliable, widely used     | Positioned as the more efficient successor                                   |

---

On paper, YOLOv11 is generally considered the stronger, more efficient architecture.

---

Results on Our Actual Dataset (254 clips, same conditions)

| Metric                    | v8s (11.17M) | v11s (9.46M) | Change (v11s vs v8s) |
|---------------------------|--------------|--------------|----------------------|
| Detections/frame (pooled) | 7.11         | 6.05         | -14.9%               |
| Mean confidence           | 0.498        | 0.486        | -2.4%                |
| Class flip rate           | 18.1%        | 16.2%        | +10.7% (better)      |
| Tracks ≤2 frames          | 42.2%        | 43.8%        | -3.7% (worse)        |
| Total detections          | 92,922       | 78,976       | -15.0%               |
| Tracked vehicles          | 7,916        | 6,858        | -13.4%               |
| Frames with 0 detections  | 253          | 170          | +32.8% (better)      |
| Unanimous class agreement | 81.9%        | 83.8%        | +2.4% (better)       |
| Untracked detections      | 7.1%         | 8.9%         | -24.6% (worse)       |

Confidence-Band Breakdown (where the lost detections actually went)

| Confidence Band | v8s count | v11s count | Change |
|-----------------|-----------|------------|--------|
| 0.0-0.2         | 4,882     | 4,872      | -0.2%  |
| 0.2-0.4         | 24,470    | 22,434     | -8.3%  |
| 0.4-0.6         | 33,158    | 27,126     | -18.2% |
| 0.6-0.8         | 28,426    | 23,479     | -17.4% |
| 0.8-1.0         | 1,986     | 1,065      | -46.4% |

This is the key finding: v11s isn't pruning low-confidence noise — it's losing its **highest-confidence, most reliable detections**. That's the opposite signature of a model getting "cleaner." It points to genuine missed vehicles, not filtered junk.

Testing Whether Model Size Explains the Gap (yolo11m)

| Model   | Params | Dets/frame (reference clip) | Tracked vehicles (reference clip) |
|---------|--------|-----------------------------|-----------------------------------|
| yolov8s | 11.17M | 17.42                       | 54                                |
| yolo11s | 9.46M  | 14.50                       | 44                                |
| yolo11m | 20.11M | 12.13                       | 41                                |

Even the larger yolo11m (nearly 2× the parameters of v8s) detected **fewer** vehicles, not more. This rules out "just needs more capacity" as the explanation — it points to a domain-shift issue rather than a model-size issue.

---

## Why We Chose YOLOv8

Our project's core bottleneck is **recall** — missing vehicles that are visibly present in frame (e.g. ~40 visible vehicles vs ~17 detected on a representative clip). Any model change needs to be judged primarily against this bottleneck.

- **YOLOv11 detects 15% fewer vehicles overall**, and disproportionately loses its most confident, reliable detections.
- **YOLOv11 does genuinely improve class confusion** (car↔truck mix-ups), which is a real win — but not our primary problem.
- **Scaling up to yolo11m does not recover the lost recall** — it gets worse, ruling out model capacity as the fix.

**Conclusion:** YOLOv11 is a strong, more efficient architecture in general, but for our specific dataset (older, low-resolution, degraded CCTV footage where detecting vehicles at all is the primary challenge), YOLOv8 performs better on the metric that matters most to us. We tested rather than assumed, and the data supported staying on YOLOv8 for this project.

---

*Note: This analysis compares model architectures on our specific dataset and use case. Results may differ on other datasets, particularly higher-resolution or modern footage.*